

P0337r0 | Delete operator= for polymorphic_allocator

Pablo Halpern phalpern@halpernwrightsoftware.com

2016-05-25 | Target audience: LWG & LEWG

1 Target

The problem addressed by this proposal is a defect in the Working Draft. This (very short) proposal is, therefore, targeted for C++17 and should ideally be put into the Working Draft before a CD is created, so as to avoid an unnecessary National Body comment.

2 Motivation

The `polymorphic_allocator` class template is the first stateful allocator added to the C++ working draft. As is the default for any allocator, the `propagate_on_container_copy_assignment` and `propagate_on_container_move_assignment` traits are false, meaning that the allocator does not get assigned when the container that uses the allocator gets assigned. This is intentional: most stateful allocators should be set on container construction and never changed.

Given that a `polymorphic_allocator` is not propagated on assignment, one may question the usefulness of the assignment operators. Indeed, it seems that most uses of assignment for `polymorphic_allocator` are likely to be errors. Consider the following:

```
class int_array {
    std::polymorphic_allocator<int> m_alloc;
    std::size_t m_size;
    int* m_data;
public:
    typedef std::polymorphic_allocator<int> allocator_type;
    typedef int value_type;

    int_array(std::size_t n, allocator_type alloc = {})
        : m_alloc(alloc), m_size(n), m_data(m_alloc.allocate(n)) { }

    int_array(const int_array&);
    int_array(int_array&&);

    int_array& operator=(const int_array&);
    int_array& operator=(int_array&&);

    ~int_array();
    ...
};
```

Now, let's look at the move-assignment operator. Even a knowledgeable implementer may accidentally forget that the allocator should not be moved, and write the code this way:

```
// BUGGY IMPLEMENTATION
int_array& int_array::operator=(int_array&& other) {
    m_alloc = std::move(other.m_alloc); // BAD IDEA
    m_size = other.m_size;
    m_data = other.m_data;
    other.m_size = 0;
    other.m_data = nullptr;
    return *this;
}
```

A correct implementation is as follows:

```
int_array& int_array::operator=(int_array&& other) {
    if (m_alloc != other.m_alloc)
        return *this = other; // Call copy assignment for unequal alloc

    m_size = other.m_size;
    m_data = other.m_data;
    other.m_size = 0;
    other.m_data = nullptr;
    return *this;
}
```

Polymorphic allocators are an important step towards making allocators easier to use. For this reason, we should do whatever is possible to reduce the incidence of errors like the one above. [P0339](#) extends this notion further, and proposes `polymorphic_allocator` as a vocabulary type, making non-template classes like the one above easier to write. However, P0339 is targeted for a TS whereas the change proposed here should be in C++17, *before* `polymorphic_allocator` becomes standard. Furthermore, we should set an example for stateful allocators in the future; stateful allocators for which `propagate_on_container_move/copy_assignment` is false should not have move and copy assignment operators.

3 Proposed Wording

The following wording is relative to the March 2016 Working Draft, N4582.

In section 20.11.3 [memory.polymorphic_allocator.class], change the declaration of `operator=` to be deleted instead of defaulted, as follows:

```
polymorphic_allocator& operator=(const polymorphic_allocator& rhs) = defaultdelete;
```